МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 3-4

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Брайловский Андрей Валерьевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

УСЛОВНАЯ ЛОГИКА, ПОДЗАПРОСЫ, ОБОБЩЕННЫЕ ТАБЛИЧНЫЕ ВЫРАЖЕНИЯ (CTE) И РАБОТА С JSONB В POSTGRES PRO

Цель работы: Работа направлена на формирование глубокого понимания и практического применения инструментов для реализации сложной бизнеслогики непосредственно на уровне базы данных, а также освоение работы со встроенными функциями и полуструктурированными данными.

Задание 1: использование оператора CASE

Листинг 1

```
-- 1.1. Категоризация заказов клиентов по сумме (total_amount)
SELECT
    id AS order_id,
    client_id,
    total_amount,
    CASE
        WHEN total_amount > 150000 THEN 'VIP (Свыше 150 000 руб.)'
        WHEN total_amount BETWEEN 50000 AND 150000 THEN 'Комфорт (50 000 - 150
000 руб.)'
        WHEN total_amount < 50000 AND total_amount > 0 THEN 'Бюджетный (До 50
000 руб.)'
        ELSE 'Ошибка данных или заказ без стоимости'
    END AS price_category
FROM
    customer_order
ORDER BY
    total_amount DESC;

-- 1.2. Подсчет групповых и одиночных бронирований в разрезе типов услуг
SELECT
    s.service_type,
    COUNT(CASE WHEN oi.quantity > 1 THEN 1 ELSE NULL END) AS group_bookings,
    COUNT(CASE WHEN oi.quantity = 1 THEN 1 ELSE NULL END) AS single_bookings
FROM
    service s
JOIN
    order_item oi ON s.id = oi.service_id
GROUP BY
    s.service_type;
```

	order_id	client_id	total_amount	price_category
1	4	4	210 000	VIP (Свыше 150 000 руб.)
2	9	9	183 000	VIP (Свыше 150 000 руб.)
3	2	2	120 000	Комфорт (50 000 - 150 000
4	6	6	99 000	Комфорт (50 000 - 150 000
5	1	1	85 000	Комфорт (50 000 - 150 000
6	5	5	67 000	Комфорт (50 000 - 150 000
7	10	10	56 000	Комфорт (50 000 - 150 000
8	3	3	45 000	Бюджетный (До 50 000 ру
9	8	8	32 000	Бюджетный (До 50 000 ру
10	7	7	0	Ошибка данных или зака

	service_type	group_bookings	single_bookings
1	flight	2	3
2	tour	1	5

Задание 2: использование подзапросов

Листинг 2

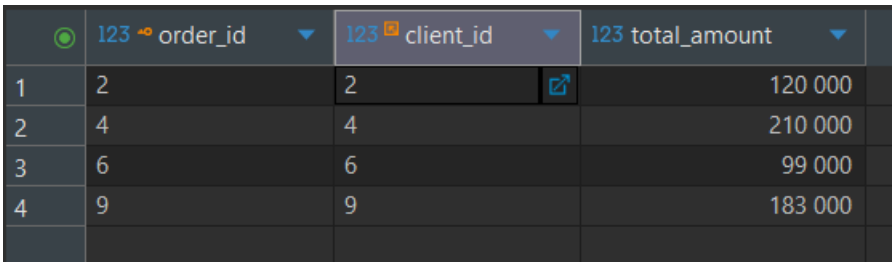
```
-- 2.1. Скалярный подзапрос: заказы выше средней суммы
SELECT
    id AS order_id,
    client_id,
    total_amount
FROM
    customer_order
WHERE
    total_amount > (SELECT AVG(total_amount) FROM customer_order);

-- 2.2. Многострочный подзапрос с IN (фильтрация по дате создания заказа)
SELECT
    (full_name).last_name,
    (full_name).first_name
FROM
    client
WHERE
    id IN (
        SELECT client_id
        FROM customer_order
        WHERE created_at >= '2025-02-01' AND created_at < '2025-03-01'
    );

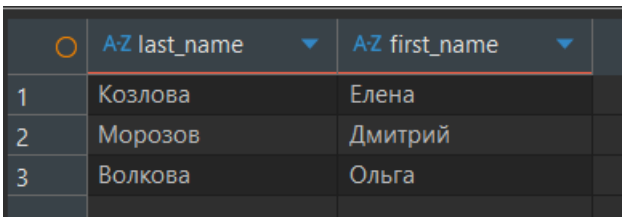
-- 2.3. Коррелированный подзапрос с EXISTS (поиск по текстовому условию в
связанной таблице)
SELECT
    (full_name).last_name,
    (full_name).first_name
FROM
    employee e
WHERE
    EXISTS (
        SELECT 1
        FROM customer_order co
```

```
        JOIN order_status os ON co.status_id = os.id
        WHERE co.employee_id = e.id
          AND os.name = 'Отменён клиентом'
    );

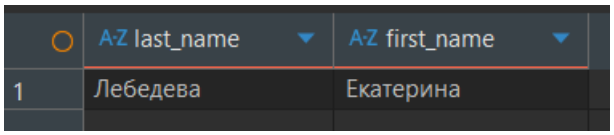
-- 2.4. Альтернативное решение задачи 2.3 с использованием JOIN
SELECT DISTINCT
    (e.full_name).last_name,
    (e.full_name).first_name
FROM
    employee e
JOIN
    customer_order co ON e.id = co.employee_id
JOIN
    order_status os ON co.status_id = os.id
WHERE
    os.name = 'Отменён клиентом';
```



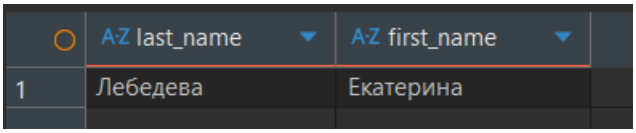
	order_id	client_id	total_amount
1	2	2	120 000
2	4	4	210 000
3	6	6	99 000
4	9	9	183 000



	last_name	first_name
1	Козлова	Елена
2	Морозов	Дмитрий
3	Волкова	Ольга



	last_name	first_name
1	Лебедева	Екатерина



	last_name	first_name
1	Лебедева	Екатерина

Задание 3: использование обобщенных табличных выражений (CTE)

Листинг 3

```
-- 3.1. Стандартное CTE: переписываем задачу 2.3
WITH cancelled_orders AS (
    -- Сначала находим ID всех сотрудников, у которых есть отмененные заказы
    SELECT DISTINCT employee_id
    FROM customer_order co
    JOIN order_status os ON co.status_id = os.id
    WHERE os.name = 'Отменён клиентом'
)
-- Теперь используем наш CTE (cancelled_orders) как обычную таблицу
```

```
SELECT
    (e.full_name).last_name,
    (e.full_name).first_name
FROM
    employee e
JOIN
    cancelled_orders c ON e.id = c.employee_id;

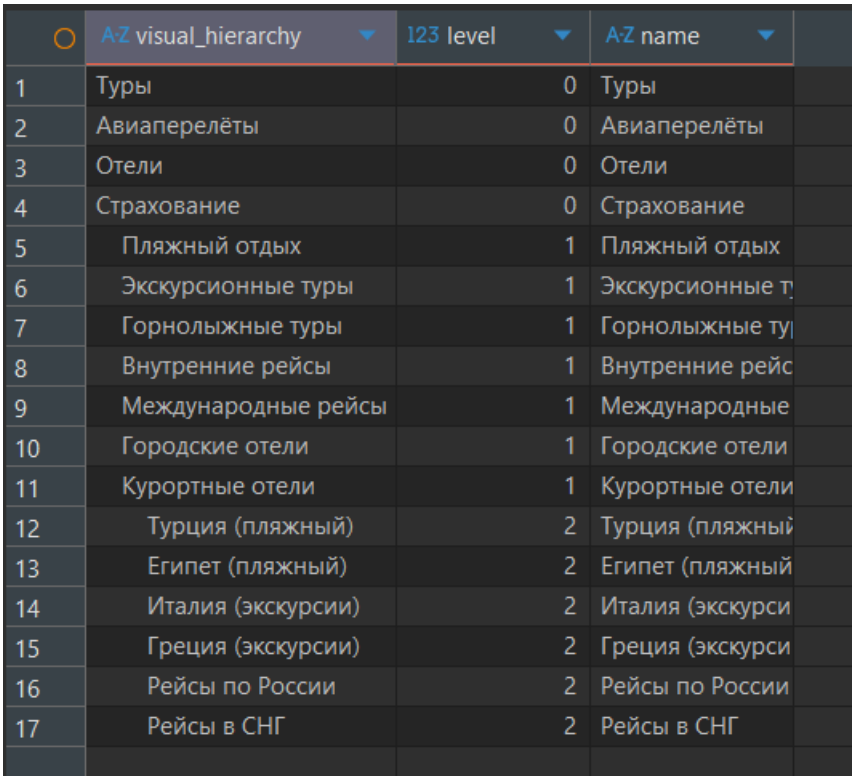
-- 3.2. Рекурсивное CTE: обход дерева категорий
WITH RECURSIVE category_tree AS (

    -- 1. Якорный (начальный) запрос: находим "корень" иерархии
    SELECT
        category_id,
        name,
        parent_id,
        0 AS level -- Устанавливаем начальный уровень 0
    FROM
        categories
    WHERE
        parent_id IS NULL -- Корневые категории (Туры, Авиаперелёты и т.д.)

    UNION ALL

    -- 2. Рекурсивный запрос: присоединяем дочерние категории
    SELECT
        c.category_id,
        c.name,
        c.parent_id,
        ct.level + 1 AS level -- Увеличиваем уровень на 1
    FROM
        categories c
    JOIN
        category_tree ct ON c.parent_id = ct.category_id
)
-- Финальная выборка с красивым форматированием
SELECT
    -- LPAD добавляет пробелы слева в зависимости от уровня (4 пробела на
уровень)
    LPAD('', level * 4, ' ') || name AS visual_hierarchy,
    level,
    name
FROM
    category_tree
ORDER BY
    level, category_id;
```

	A-Z last_name	A-Z first_name
1	Лебедева	Екатерина



	A-Z visual_hierarchy	123 level	A-Z name
1	Туры	0	Туры
2	Авиаперелёты	0	Авиаперелёты
3	Отели	0	Отели
4	Страхование	0	Страхование
5	Пляжный отдых	1	Пляжный отдых
6	Экскурсионные туры	1	Экскурсионные т
7	Горнолыжные туры	1	Горнолыжные ту
8	Внутренние рейсы	1	Внутренние рейс
9	Международные рейсы	1	Международные
10	Городские отели	1	Городские отели
11	Курортные отели	1	Курортные отели
12	Турция (пляжный)	2	Турция (пляжный
13	Египет (пляжный)	2	Египет (пляжный
14	Италия (экскурсии)	2	Италия (экскурси
15	Греция (экскурсии)	2	Греция (экскурси
16	Рейсы по России	2	Рейсы по России
17	Рейсы в СНГ	2	Рейсы в СНГ

Задание 4: Встроенные функции при работе с данными

Листинг 4

```
-- 4.1. Демонстрация строковых функций для работы с данными клиентов
SELECT
    -- CONCAT: Формирование единой строки ФИО для вывода в печатные формы и
договоры
    CONCAT((full_name).last_name, ' ', (full_name).first_name, ' ',
(full_name).middle_name) AS formatted_fio,

    -- UPPER и SUBSTRING: Маскирование логина (первые 3 буквы заглавными +
звездочки) для безопасности при отображении в интерфейсе
    CONCAT(UPPER(SUBSTRING(login FROM 1 FOR 3)), '***') AS masked_login,

    -- LENGTH: Подсчет количества символов в email (используется для валидации
и ограничения длины поля)
    LENGTH(email) AS email_chars_count
FROM
    client
LIMIT 5;

-- 4.2. Демонстрация функций работы с датой и временем для аналитики
SELECT
    (full_name).last_name,
    birth_date,

    -- AGE: Расчёт точного возраста клиента на текущий день (в годах, месяцах
и днях) для проверки совершеннолетия или сегментации
    AGE(CURRENT_DATE, birth_date) AS exact_client_age,

    -- EXTRACT: Извлечение номера месяца из даты создания заказа для
построения графиков сезонности
    EXTRACT(MONTH FROM created_at) AS order_month_number,
```

```
    -- DATE_TRUNC: Округление даты создания заказа до начала месяца (удобно
для группировки в финансовых отчетах)
    DATE_TRUNC('month', created_at) AS order_month_start
FROM
    client c
JOIN
    customer_order co ON c.id = co.client_id
LIMIT 5;

-- 4.3. Демонстрация математических функций для финансовых расчетов
SELECT
    id AS order_id,
    total_amount,

    -- ROUND: Математическое округление итоговой суммы заказа до целого рубля
(для красивого отображения в SMS и чеках без копеек)
    ROUND(total_amount) AS rounded_total,

    -- CEIL: Округление в большую сторону суммы агентского вознаграждения
(например, 2.5% от заказа) до целого рубля
    CEIL(total_amount * 0.025) AS agent_fee_ceil,

    -- ABS: Получение абсолютного значения (бизнес-логика: отображение суммы
возврата средств клиенту без знака "минус")
    ABS(-5000.00) AS abs_refund_example
FROM
    customer_order
LIMIT 5;
```

	formatted_fio	masked_login	email_chars_count
1	Иванов Иван Иванович	IVA***	14
2	Петрова Анна Сергеевна	ANN***	20
3	Сидоров Алексей Владим	ALE***	18
4	Козлова Елена Дмитриев	LEN***	21
5	Морозов Дмитрий Алекс	DMO***	15

	last_name	birth_date	exact_client_age	order_month_number	order_month_start
1	Иванов	1990-07-15	36 years 2 mons 9 days	1	2025-01-01 00:00:00.000 +0300
2	Петрова	1988-12-01	37 years 9 mons 23 days	1	2025-01-01 00:00:00.000 +0300
3	Сидоров	1985-03-22	41 years 6 mons 2 days	1	2025-01-01 00:00:00.000 +0300
4	Козлова	1995-08-11	31 years 1 mon 13 days	2	2025-02-01 00:00:00.000 +0300
5	Морозов	2000-11-05	25 years 10 mons 19 days	2	2025-02-01 00:00:00.000 +0300

	order_id	total_amount	rounded_total	agent_fee_ceil	abs_refund_example
1	1	85 000	85 000	2 125	5 000
2	2	120 000	120 000	3 000	5 000
3	3	45 000	45 000	1 125	5 000
4	4	210 000	210 000	5 250	5 000
5	5	67 000	67 000	1 675	5 000

Задание 5: Работа с неструктурированными данными (JSONB)

Листинг 5

```
-- 5.1. Создание таблицы логов и наполнение тестовыми JSONB-данными
CREATE TABLE IF NOT EXISTS user_logs (
    log_id SERIAL PRIMARY KEY,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    user_id INT,
    event_data JSONB
);

INSERT INTO user_logs (user_id, event_data) VALUES
(1, '{"event": "login", "browser": "Chrome", "os": "Windows", "success":
true}'),
(2, '{"event": "view_tour", "tour_id": 5, "duration_sec": 120}'),
(1, '{"event": "booking", "tour_id": 5, "amount": 85000, "success": true}'),
(3, '{"event": "login", "browser": "Safari", "os": "iOS", "success": false}'),
(4, '{"event": "view_tour", "tour_id": 2, "duration_sec": 45}'),
(2, '{"event": "booking", "tour_id": 2, "amount": 120000, "success": true}'),
(5, '{"event": "login", "browser": "Firefox", "os": "Linux", "success":
true}');

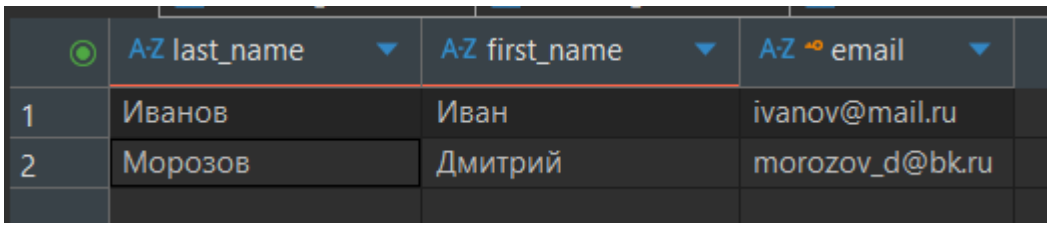
-- 5.2. Фильтрация по JSONB внутри CTE с последующим JOIN к таблице клиентов
WITH successful_logins AS (
    -- Оператор @> проверяет, содержит ли event_data подмножество {"event":
"login", "success": true}
    SELECT DISTINCT user_id
    FROM user_logs
    WHERE event_data @> '{"event": "login", "success": true}'
)
SELECT
    (c.full_name).last_name,
    (c.full_name).first_name,
    c.email
FROM
    client c
JOIN
    successful_logins sl ON c.id = sl.user_id;

-- 5.3. Массовое обновление: добавляем новое поле "source" во все JSON-
документы
UPDATE user_logs
SET event_data = event_data || '{"source": "web"}'::jsonb;

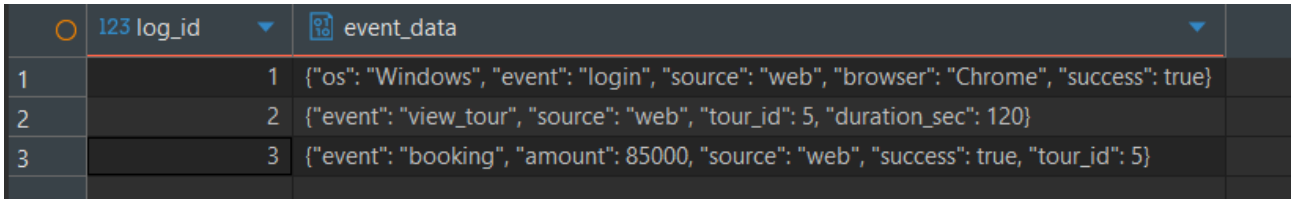
-- Проверяем результат
SELECT log_id, event_data FROM user_logs LIMIT 3;

-- 5.4. Точечное изменение: меняем значение ключа 'success' на false для
конкретной записи
UPDATE user_logs
SET event_data = jsonb_set(
    event_data,             -- 1. Исходное поле
    '{success}',            -- 2. Путь к ключу в виде массива строк
    'false'::jsonb          -- 3. Новое значение (обязательно приводим к типу
jsonb!)
)
WHERE log_id = 3;

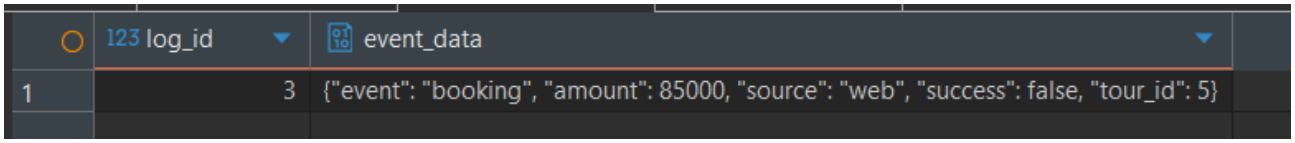
-- Проверяем, что изменилось только нужное поле
SELECT log_id, event_data FROM user_logs WHERE log_id = 3;
```



	last_name	first_name	email
1	Иванов	Иван	ivanov@mail.ru
2	Морозов	Дмитрий	morozov_d@bk.ru



	log_id	event_data
1	1	{"os": "Windows", "event": "login", "source": "web", "browser": "Chrome", "success": true}
2	2	{"event": "view_tour", "source": "web", "tour_id": 5, "duration_sec": 120}
3	3	{"event": "booking", "amount": 85000, "source": "web", "success": true, "tour_id": 5}



	log_id	event_data
1	3	{"event": "booking", "amount": 85000, "source": "web", "success": false, "tour_id": 5}

АНАЛИТИЧЕСКИЕ ЗАПРОСЫ: ОКОННЫЕ ФУНКЦИИ И ПОСТРОЕНИЕ СВОДНЫХ ТАБЛИЦ

Цель работы: Целью данной практической работы является формирование у студентов углубленных навыков работы со сложными аналитическими запросами в СУБД PostgreSQL.

Задание №1: использование ранжирующих функций

Листинг 1

```
-- Задание №1: Ранжирование заказов по сумме внутри каждого менеджера
WITH RankedOrders AS (
    SELECT
        -- Формируем ФИО менеджера из составного типа fio_type
        (e.full_name).last_name || ' ' || (e.full_name).first_name AS
manager_name,
        co.id AS order_id,
        co.total_amount,
        -- Ранжируем заказы по убыванию суммы в пределах каждого менеджера
        DENSE_RANK() OVER (PARTITION BY e.id ORDER BY co.total_amount DESC) AS
order_rank
    FROM employee e
    JOIN customer_order co ON e.id = co.employee_id
)
SELECT
    manager_name AS "Менеджер",
    order_id AS "ID Заказа",
    total_amount AS "Сумма заказа",
    order_rank AS "Ранг"
FROM RankedOrders
WHERE order_rank <= 3 -- Оставляем только топ-3
ORDER BY manager_name, order_rank;
```

	Менеджер	ID Заказа	Сумма заказа	Ранг
1	Лебедева Екатерина	3	45 000	1
2	Лебедева Екатерина	7	0	2
3	Новиков Игорь	4	210 000	1
4	Новиков Игорь	8	32 000	2
5	Попов Александр	9	183 000	1
6	Попов Александр	2	120 000	2
7	Попов Александр	6	99 000	3
8	Смирнов Андрей	1	85 000	1
9	Смирнов Андрей	5	67 000	2
10	Смирнов Андрей	10	56 000	3

Задание №2: использование агрегатных оконных функций

Листинг 2

```
-- Задание №2: Нарастающий итог продаж по месяцам
WITH MonthlyOrders AS (
    -- Сначала агрегируем продажи по месяцам
    SELECT
        DATE_TRUNC('month', created_at)::DATE AS order_month,
        SUM(total_amount) AS monthly_amount
    FROM customer_order
    GROUP BY order_month
)
SELECT
    order_month AS "Месяц",
    monthly_amount AS "Сумма за месяц",
    -- Оконная функция считает нарастающий итог от первого месяца до текущего
    SUM(monthly_amount) OVER (ORDER BY order_month) AS "Нарастающий итог"
FROM MonthlyOrders
ORDER BY order_month;
```

	Месяц	Сумма за месяц	Нарастающий итог
1	2025-01-01	250 000	250 000
2	2025-02-01	376 000	626 000
3	2025-03-01	271 000	897 000

Задание №3: использование функции смещения

Листинг 3

```
-- Задание №3: Сравнение продаж с предыдущим месяцем
WITH TotalMonthlySales AS (
    SELECT
        DATE_TRUNC('month', created_at)::DATE AS sale_month,
        SUM(total_amount) AS total_amount
    FROM customer_order
    GROUP BY sale_month
)
SELECT
    sale_month AS "Месяц",
    total_amount AS "Продажи за текущий месяц",
    -- Получаем значение из предыдущей строки (смещение 1), если нет -
подставляем 0
    LAG(total_amount, 1, 0) OVER (ORDER BY sale_month) AS "Продажи за
предыдущий месяц",
    total_amount - LAG(total_amount, 1, 0) OVER (ORDER BY sale_month) AS
"Изменение"
FROM TotalMonthlySales
ORDER BY sale_month;
```

	Месяц	Продажи за текущий месяц	Продажи за предыдущий месяц	Изменение
1	2025-01-01	250 000	0	250 000
2	2025-02-01	376 000	250 000	126 000
3	2025-03-01	271 000	376 000	-105 000

Задание №4: построение сводной таблицы

Листинг 4

```
-- Задание №4: Сводная таблица продаж туроператоров по кварталам
SELECT
    t_op.name AS "Туроператор",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 1 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q1",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 2 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q2",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 3 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q3",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 4 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q4"
FROM customer_order co
JOIN order_item oi ON co.id = oi.order_id
JOIN service s ON oi.service_id = s.id
JOIN tour_operator t_op ON s.tour_operator_id = t_op.id
GROUP BY t_op.name
ORDER BY t_op.name;
```

	A-Z Туроператор	123 Q1	123 Q2	123 Q3	123 Q4
1	ANEX Tour	210 000	0	0	0
2	Biblio Globus	32 000	0	0	0
3	Coral Travel	85 000	0	0	0
4	DSBW	183 000	0	0	0
5	Intourist	99 000	0	0	0
6	Mouzenidis Travel	183 000	0	0	0
7	Pegas Touristik	67 000	0	0	0
8	Sunmar	45 000	0	0	0
9	Tez Tour	176 000	0	0	0
10	Русский Экспресс	56 000	0	0	0

Задание №5: создание модифицируемого представления

Листинг 5

```
-- Задание №5: Создание модифицируемого представления для 5-звездочных отелей
CREATE OR REPLACE VIEW five_star_hotels AS
SELECT
    id,
    name,
    country_id,
    city_id,
    stars,
    full_address,
    description,
    meal_type_id
FROM hotel
WHERE stars = 5;
```

Задание №6: модификация данных через представление

Листинг 6

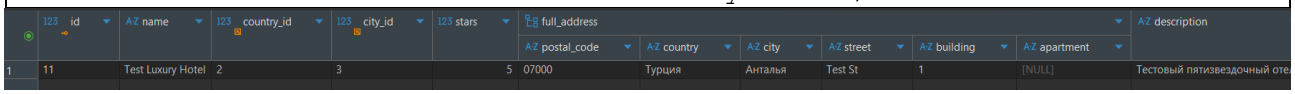
```
-- Задание №6: Модификация данных через представление

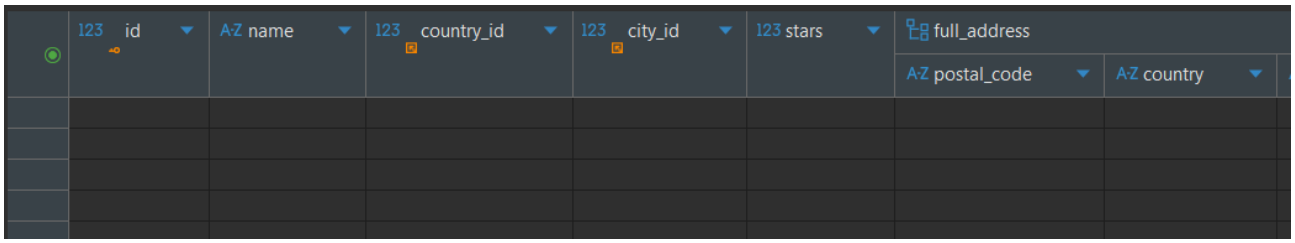
-- 1. Добавляем новую запись через представление
INSERT INTO five_star_hotels (name, country_id, city_id, stars, full_address,
description, meal_type_id)
VALUES (
    'Test Luxury Hotel',
    (SELECT id FROM country WHERE code = 'TR'),
    (SELECT id FROM city WHERE iata_code = 'AYT'),
    5,
    ROW('07000', 'Турция', 'Анталья', 'Test St', '1', NULL), -- Составной тип
address_type
    'Тестовый пятизвездочный отель',
    (SELECT id FROM meal_type WHERE code = 'AI')
);

-- Проверяем, что запись добавилась в базовую таблицу
SELECT * FROM hotel WHERE name = 'Test Luxury Hotel';

-- 2. Удаляем запись через представление
DELETE FROM five_star_hotels WHERE name = 'Test Luxury Hotel';

-- Проверяем, что запись удалилась из базовой таблицы
SELECT * FROM hotel WHERE name = 'Test Luxury Hotel';
```





Задание №7: создание немодифицируемого аналитического представления

Листинг 7

```
-- Задание №7: Немодифицируемое аналитическое представление по туроператорам
CREATE OR REPLACE VIEW tour_operator_analytics AS
SELECT
    t_op.name AS tour_operator_name,
    COUNT(DISTINCT s.id) AS total_services,
    COALESCE(SUM(oi.price * oi.quantity), 0) AS total_revenue,
    -- Условная агрегация: считаем только полностью оплаченные заказы
    COUNT(DISTINCT co.id) FILTER (WHERE os.name = 'Оплачен полностью') AS
paid_orders_count
FROM tour_operator t_op
LEFT JOIN service s ON t_op.id = s.tour_operator_id
LEFT JOIN order_item oi ON s.id = oi.service_id
LEFT JOIN customer_order co ON oi.order_id = co.id
LEFT JOIN order_status os ON co.status_id = os.id
```

```
GROUP BY t_op.name;
```

Задание №8: использование аналитического представления в запросах

Листинг 8

```
-- Задание №8: Фильтрация данных аналитического представления
SELECT
    tour_operator_name AS "Туроператор",
    total_revenue AS "Общая выручка",
    total_services AS "Кол-во услуг",
    paid_orders_count AS "Оплаченные заказы"
FROM tour_operator_analytics
WHERE total_revenue > 100000 -- Отбираем только успешных туроператоров
ORDER BY total_revenue DESC;
```

	Туроператор	Общая выручка	Кол-во услуг	Оплаченные заказы
1	ANEX Tour	210 000	1	0
2	DSBW	183 000	1	0
3	Mouzenidis Travel	183 000	1	0
4	Tez Tour	176 000	1	0

Задание №9: Создание и обновление материализованного представления

Листинг 9

```
-- Задание №9: Материализованное представление для трат клиентов

-- 1. Создаем материализованное представление
CREATE MATERIALIZED VIEW client_total_spent AS
SELECT
    c.id AS client_id,
    (c.full_name).last_name || ' ' || (c.full_name).first_name AS client_name,
    COUNT(co.id) AS orders_count,
    COALESCE(SUM(co.total_amount), 0) AS total_spent
FROM client c
LEFT JOIN customer_order co ON c.id = co.client_id
GROUP BY c.id, c.full_name;

-- Проверяем исходные данные
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- 2. Добавляем новый заказ для клиента Иванов И.И. в исходную таблицу
INSERT INTO customer_order (created_at, client_id, employee_id, status_id,
paid_at, total_amount)
VALUES (
    CURRENT_TIMESTAMP,
    (SELECT id FROM client WHERE login = 'ivan_i'),
    (SELECT id FROM employee WHERE login = 'a_smirnov'),
    (SELECT id FROM order_status WHERE name = 'Новый'),
    NULL,
    50000.00
);
```

```sql
-- 3. Проверяем материализованное представление (данные еще НЕ обновились, так как оно хранит снимок)
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- 4. Выполняем ручное обновление материализованного представления
REFRESH MATERIALIZED VIEW client_total_spent;

-- 5. Снова проверяем (теперь сумма total_spent увеличилась на 50000)
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- --- Очистка тестовых данных (чтобы не засорять БД) ---
DELETE FROM customer_order
WHERE total_amount = 50000.00
  AND client_id = (SELECT id FROM client WHERE login = 'ivan_i')
  AND created_at >= CURRENT_DATE;
REFRESH MATERIALIZED VIEW client_total_spent;

-- 6. Проверяем удаление
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';
```

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	2	135 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000